소속 변경과 발주 승인

업무 설명용 합성 시나리오입니다. 판정과 함께 적용한 규칙, 사용한 근거, 재검토 대상을 제시하는 것이 목표입니다.

"검토자의 소속 본부가 바뀌었습니다. 이 발주를 그대로 승인해도 되나요?"

구매 담당자

검토자 소속이 바뀌었는데
이 발주를 진행해도 될까?

승인 담당자

자격과 소속 관계에
바뀐 규정도 적용됐을까?

감사 담당자

어떤 기록과 규정 버전으로
그렇게 판단했을까?

발표 구성

업무의 질문

왜 이 판단이 필요한가
어떤 근거를 설명해야 하나

작은 부품 예제

표현 → 추론 → 검사
각 도구의 역할 구분

우리 연구의 설계

기존 결과와 남은 질문
공정한 비교와 사례 구성

- **업무 질문:** 판단에 필요한 데이터와 의미
- **제조 부품 예제:** 표현·추론·검증의 차이
- **연구 결과:** 검사한 입력·조건·출력
- **후속 설계:** 실제 자료의 한계와 데모 화면

데이터의 의미·연결·시점

> **근거**는 판단에 사용한 기록이고 **시맨틱**은 그 기록의 뜻과 관계입니다. 같은 status라도 계약·승인·자격에서 의미가 다릅니다.



- 키로 연결: 부품 P1 → 계약 C7 → 검토자 R2 → 소속 이력
- 시점 확인: 승인 당시 소속과 현재 소속이 다르면 어느 규칙을 적용할지 결정

설명용 합성 레코드 · 이력은 개념 예시이며 실제 실행 입력은 한 시점의 스냅샷

규칙 변경의 담당자와 요구사항

> 기존 Python·SQL·룰 엔진·DAG로도 구현할 수 있습니다. 연구의 관심은 변경된 의미와 규칙을 여러 소비처에 일관되게 전달하는 방법입니다.

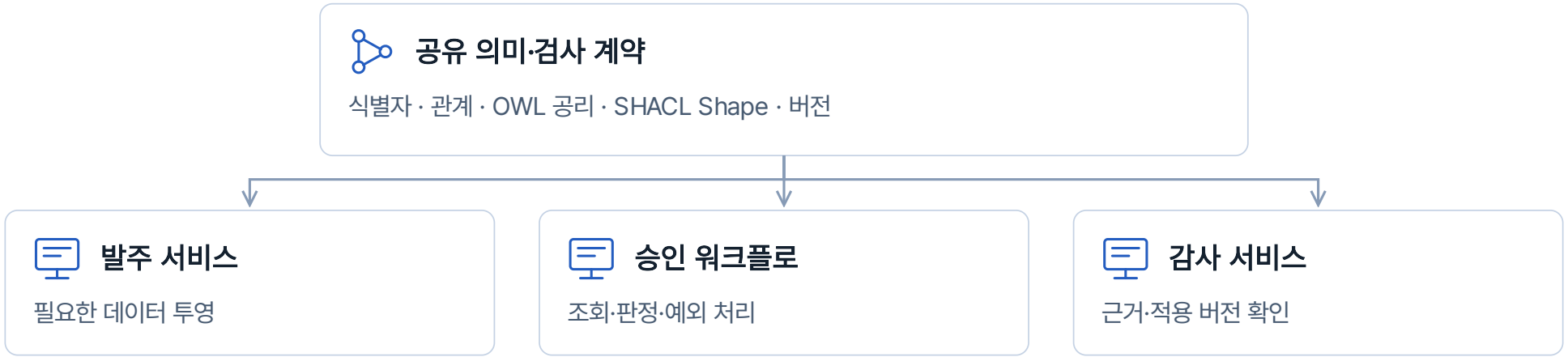
담당자별로 다른 변경 판단 근거



- 공유 JSON·스키마 레지스트리·공통 라이브러리도 비교 대상
- 표준 형식 도입만으로 변경 비용과 배포 문제가 사라지지는 않음

공유 의미와 검사 계약

> 연구 의도는 의미와 검사 조건을 공통 자산으로 관리하는 것입니다. 매핑, 버전 배포, 접근 제어와 회귀 테스트는 별도로 구현해야 합니다.



구성	공유하려는 내용
RDF	개체의 식별자와 관계 표현
OWL	클래스·관계의 공리와 함의의 기준
SHACL / Shape	검사 대상과 조건을 담은 규칙 구조
pySHACL	Shape를 실행해 표준 검증 보고서 생성

출처: RDF · OWL · SHACL · pySHACL

추론·검증·사실 확인의 구분

> 입력이 틀리면 논리적으로 일관된 결론도 현실에서는 틀릴 수 있습니다.

추론

정의와 기록으로부터
어떤 결론이 따라오는가

→

데이터 검증

제출한 데이터가
정해 둔 조건을 만족하는가

→

현실의 사실 확인

원천 근거·현장·전문가로
입력과 결론을 확인

작업	묻는 질문	필요한 근거
추론	정의와 기록에서 어떤 결론이 따라오는가	입력 사실 + 공리·규칙
데이터 검증	제출한 데이터가 조건을 만족하는가	검사할 그래프 + Shape
현실의 사실 확인	기록과 결론이 실제 업무에서 맞는가	독립 자료·현장 확인·전문가 판단

RDF·RDFS·OWL·SHACL의 역할

> 언어·정의와 실행 도구를 구분합니다. 파일을 읽는 것만으로 추론이 켜지지는 않습니다.

구성	역할	부품 P1 예
RDF	개체·관계를 트리플로 표현	P1은 조립품 A1에 장착된다
RDFS	클래스 포함 관계, domain/range 등 정의	볼트는 부품, 부품은 구성 부품
OWL	더 풍부한 논리적 공리 표현	역관계, 클래스 조합·배타성, 동일성
추론기	지원하는 공리·규칙의 결론 계산	P1도 구성 부품이라는 타입 도출
SHACL	주어진 RDF의 조건 충족 검사	제조사 식별자가 기록됐는가

출처: RDF · RDFS · OWL · SHACL

RDF 트리플과 제조 부품

> RDF는 개체와 관계를 식별자로 연결합니다. Turtle은 그 그래프를 텍스트로 적는 형식입니다.

관계 하나를 주어 · 술어 · 목적어로 기록



ex: = https://example.org/manufacturing/

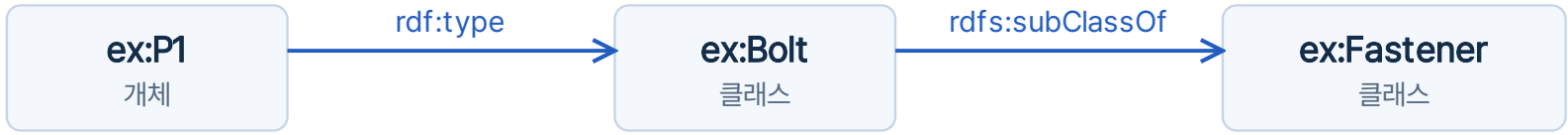
```
@prefix ex: <https://example.org/manufacturing/> .
ex:P1 a ex:Bolt ;                # P1은 볼트 개체
    ex:installedIn ex:A1 .       # P1은 조립품 A1에 장착
```

- 입력은 설명용 합성 부품 기록이며 실제 설치 사실 확인과는 별개

개체·클래스·포함 관계

> P1은 하나의 부품이고 Bolt는 그 부품이 속한 클래스입니다. 클래스 간 포함 관계와 개체의 소속을 구분합니다.

개체의 타입과 클래스의 포함 관계



rdf:type은 개체 → 클래스, rdfs:subClassOf는 클래스 → 클래스 관계다.

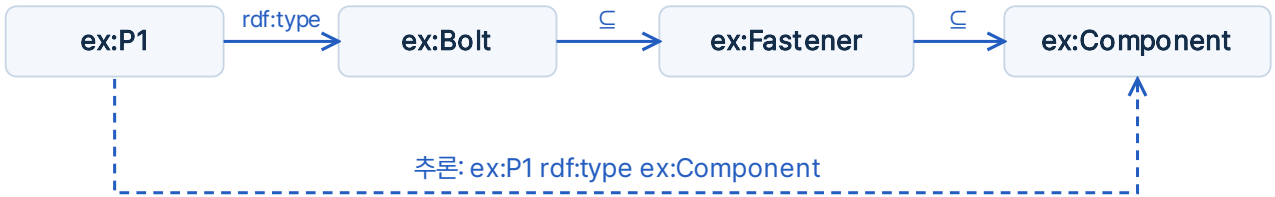
```
ex:P1 a ex:Bolt .                          # 개체 → 클래스
ex:Bolt rdfs:subClassOf ex:Fastener .      # 클래스 → 상위 클래스
```

출처: RDFS 클래스와 인스턴스

RDFS 클래스 계층과 타입 추론

> 하위 클래스의 개체는 상위 클래스에도 속합니다. 점선은 정의를 적용해 얻는 추가 결론입니다.

클래스 포함 관계에서 개체 타입 도출



⊆ = rdfs:subClassOf · 실선은 명시한 관계, 점선은 규칙으로 도출한 관계

- 입력: `P1 a Bolt` + `Bolt ⊆ Fastener ⊆ Component`
- 출력: `P1 a Fastener`, `P1 a Component`
- 실제 부품의 정체가 맞는지는 원천 자료로 별도 확인

출처: RDFS

domain·range의 타입 해석

> domain과 range는 관계의 주어·목적어를 어떤 타입으로 해석할지 정합니다. 입력을 거부하는 검증 규칙과는 다릅니다.

domain · range는 관계 양끝의 타입을 도출



타입 추론 규칙이다. 허용 타입을 검사하는 입력 검증 규칙과 구분한다.

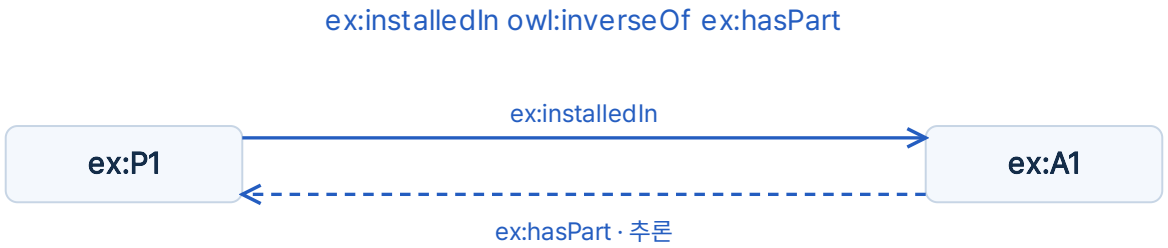
```
ex:installedIn rdfs:domain ex:Component ; # 주어는 구성 부품
               rdfs:range ex:Assembly . # 목적어는 조립품
```

출처: RDFS domain/range

OWL 공리와 역관계

> OWL의 공리는 관계와 클래스의 논리적 의미를 명시합니다. 실제 계산 범위는 선택한 추론기의 지원 기능에 달려 있습니다.

역속성 정의로 반대 방향 관계 도출



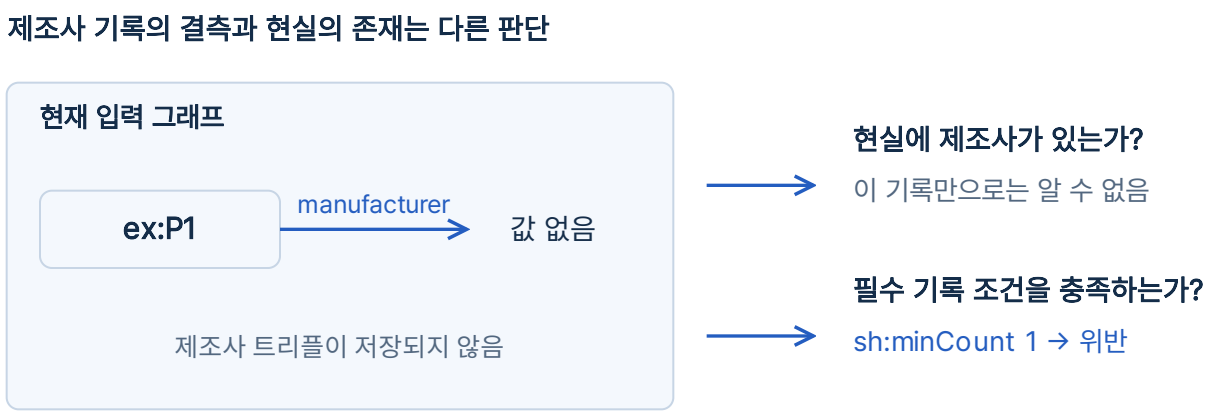
역방향 관계는 owl:inverseOf 선언이 있을 때 도출한다.

- `inverseOf` : P1이 A1에 장착되면 A1은 P1을 포함
- `equivalentClass` : “인증 부품이면서 체결 부품” 같은 분류 정의
- `disjointWith` / `sameAs` : 배타 분류 / 동일 개체 선언

출처: OWL 2 Primer

열린 세계와 기록 부재

> 제조사 기록이 없다는 사실만으로 제조사가 존재하지 않는다고 결론 내릴 수 없습니다. 제출 자료의 필수 항목 검사는 별도로 선언합니다.



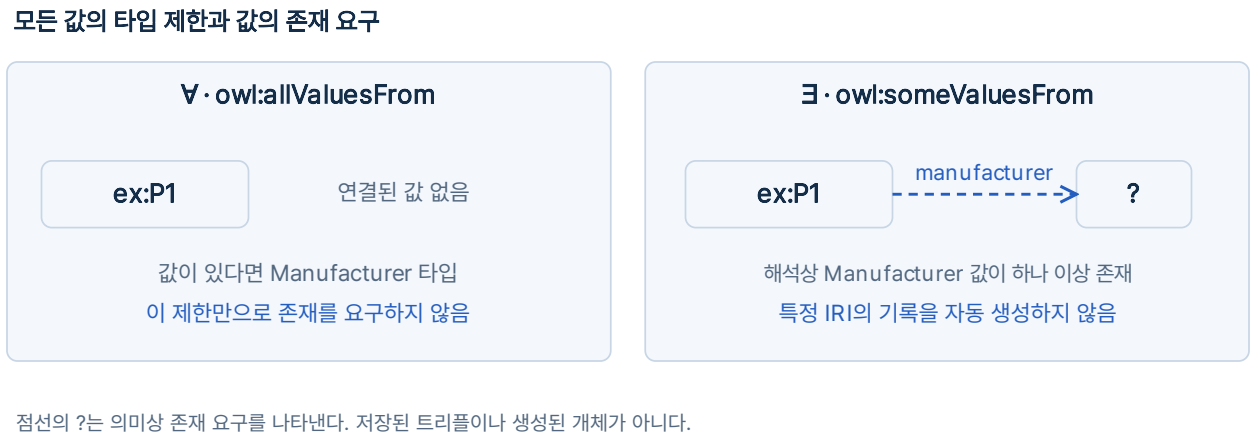
OWL의 열린 세계 해석과 SHACL의 입력 그래프 검사를 구분한다.

- OWL: 추가 근거가 없으면 제조사에 대한 판단을 유보
- SHACL: 제출된 P1 기록에 제조사 IRI가 없으면 `minCount` 위반

출처: OWL 열린 세계 가정

전체 조건·존재 조건·동일성

> “모두 인증 부품”이라는 조건은 연결 대상의 존재를 보장하지 않습니다. 식별자 두 개도 서로 다른 실체 둘을 자동으로 뜻하지 않습니다.



점선의 ?는 의미상 존재 요구를 나타낸다. 저장된 트리플이나 생성된 개체가 아니다.

- `allValuesFrom`: 연결된 대상이 있다면 지정 클래스에 속함
- `someValuesFrom`: 지정 클래스에 속한 대상의 존재를 요구
- 다른 회사임을 명시하려면 별도 동일성·차이 선언과 근거 검토

출처: OWL 제약과 동일성

추론으로 추가되는 조회 결과

추론을 쓰는 목적: 직접 입력하지 않은 결론도 정의를 따라 찾아내는 것

기록과 정의

입력 기록

P1은 볼트다.
P1은 조립품 A1에 장착된다.

함께 제공한 정의

볼트 ⊆ 체결 부품 ⊆ 구성 부품
장착된다 ↔ 포함한다 (역관계)

추론 후 추가로 조회할 수 있는 결론

P1은 구성 부품이다.

A1은 P1을 포함한다.

직접 적지 않은 타입과 역방향 관계를
입력과 정의를 결합해 계산한다.

- 다음 장에서는 같은 입력에 RDFS·OWL-RL을 적용한 실제 조회 결과를 비교한다.

원본·RDFS·OWL-RL의 조회 결과

> 같은 입력이라도 적용한 정의와 조회 범위에 따라 보이는 결론이 달라집니다. 제조사 기록은 세 조건 모두 비어 있습니다.

저장된 조회 결과 · 없음은 현실의 거짓을 뜻하지 않음

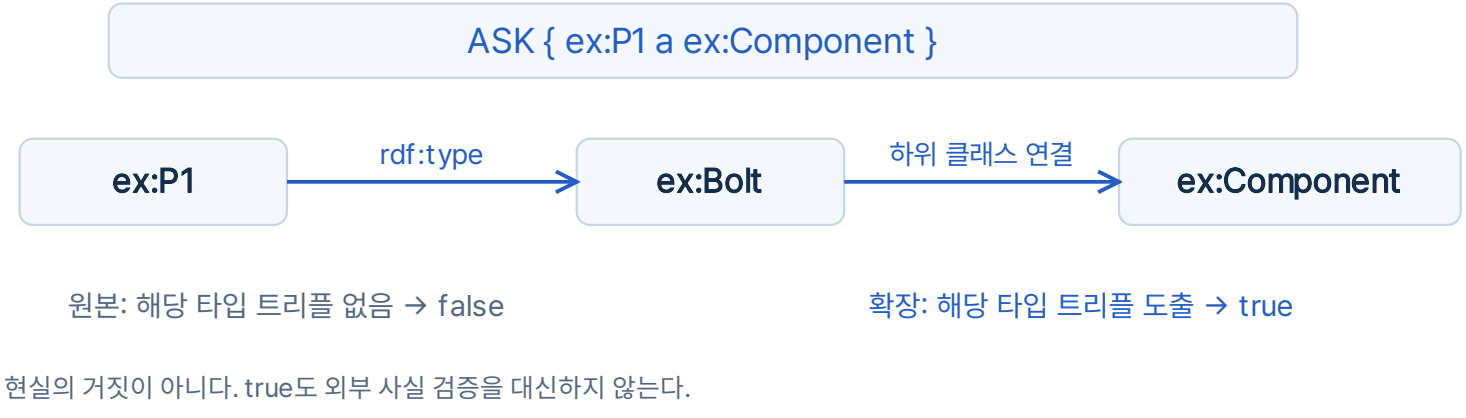
조회 결과	원본	RDFS	OWL-RL
P1 a Component	없음	있음	있음
A1 a Assembly	없음	있음	있음
A1 hasPart P1	없음	없음	있음
제조사 기록	없음	없음	없음

- 입력: P1의 볼트 타입, A1 장착 관계, 클래스·역관계 정의
- 출력: 부품 타입·조립품 타입·역관계의 존재 여부
- 버튼은 실제 저장 결과를 비교하며 새 Python 실행을 시작하지 않음

조회 실패와 현실의 거짓

> 단순 ASK의 false는 선택한 그래프에서 해당 패턴을 찾지 못했다는 뜻입니다. 현실에서 거짓이라는 판정과 구분합니다.

ASK 결과는 선택한 그래프의 패턴 일치 여부



false는 현실의 거짓이 아니다. true도 외부 사실 검증을 대신하지 않는다.

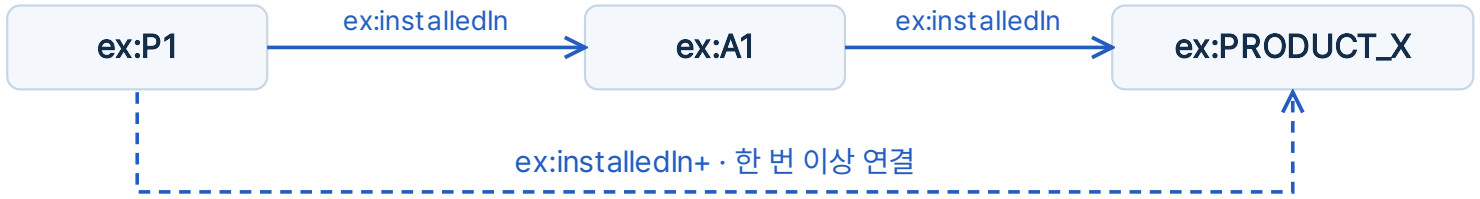
```
ASK { ex:P1 a ex:Component }  # P1의 Component 타입이 조회되는가
```

- 원본 false → 확장본 true: 새 사실을 외부에서 확인한 것이 아니라 정의를 적용한 결과

다단계 관계의 고정점 계산

> 여러 관계를 합쳐 도달 가능한 대상 쌍을 계산합니다. 가능한 모든 경로를 나열하는 작업과는 다릅니다.

경로 탐색은 연결된 시작점 · 끝점 쌍을 반환



반환 예: (P1, A1), (P1, PRODUCT_X) · 통과한 노드 목록이나 경로 수를 뜻하지 않는다.

```
edge(P1, A1), edge(A1, PRODUCT_X)
reach(X, Y) ← edge(X, Y)                    # 직접 연결도 도달
reach(X, Z) ← reach(X, Y), edge(Y, Z)       # 도달 관계를 연장
```

- 실제 Python 집합 계산의 추가 결과: `(P1, PRODUCT_X)`

SKOS 용어 분류와 OWL 클래스

> 용어집의 상하 개념, 클래스 포함 관계, 개체의 타입은 서로 다른 관계입니다. 같은 "볼트"라는 말도 모델링 역할을 구분합니다.

개념 분류와 클래스 분류의 서로 다른 연결



skos:broader와 rdfs:subClassOf는 다른 의미다. broader 자체를 추이적 속성으로 읽지 않는다.

- SKOS: 용어·라벨·분류 체계의 연결
- OWL/RDFS: 개체가 어떤 클래스에 속하는지와 논리적 관계 정의

출처: SKOS Primer · SKOS Reference

SHACL의 입력과 검사 결과

> SHACL 통과는 선언한 조건의 충족입니다. 제조사 정보가 현실에서 참이라는 인증은 아닙니다.



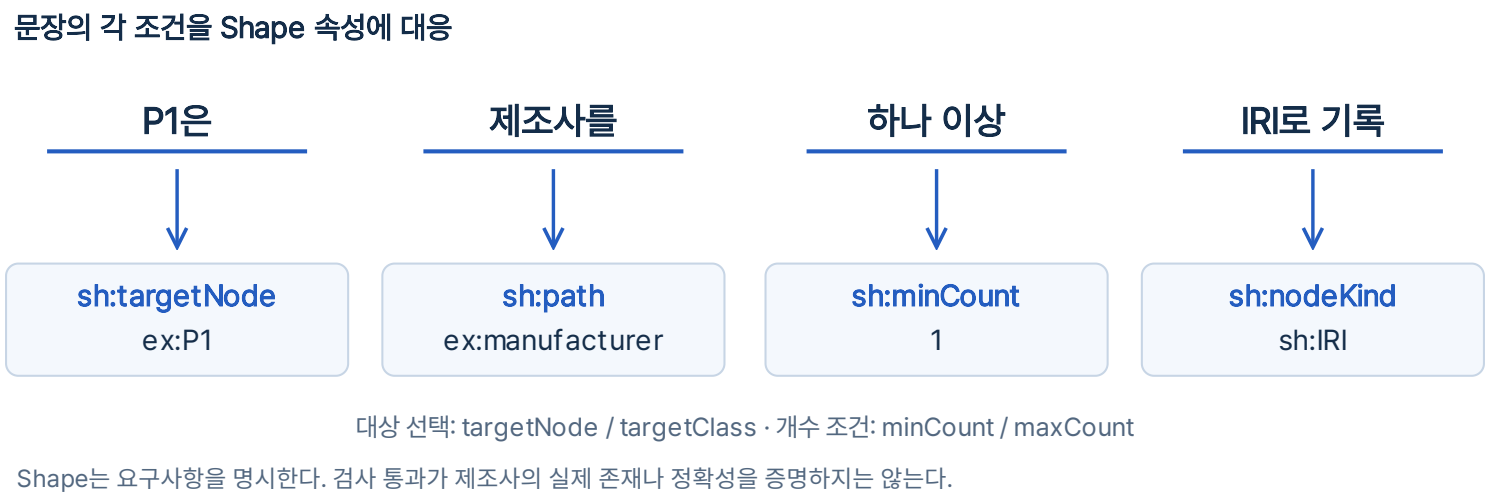
SHACL: Shapes Constraint Language. **Shape**: 검사 대상과 조건을 묶은 구조.

- Data Graph: 검사할 부품 기록
- Shapes Graph: “P1에 제조사 IRI가 하나 이상 있어야 한다”
- Validation Report: 어떤 개체·속성이 어떤 조건을 어겼는지 기록

출처: SHACL

업무 문장의 Shape 변환

> Shape 작성은 요구 문장을 검사 대상·속성·개수·값 종류로 나누는 과정입니다. 뜻이 맞는지는 정상·위반·결측 반례로 확인합니다.



- `targetNode`는 특정 개체, `targetClass`는 클래스의 대상 개체를 선택
- 문법이 맞는 Shape도 잘못된 업무 조건을 검사할 수 있음

LLM으로 RDF·Shape 초안 작성

① 문서와 요구 조건을 프롬프트로 전달

입력

문서: “볼트 P1은 조립품 A1에 장착된다.”

업무 조건: “P1의 제조사를 IRI로 하나 이상 기록해야 한다.”

프롬프트 예시

문서의 사실을 `data.ttl`로, 업무 조건을 SHACL 문법의 `shapes.ttl`로 작성해 줘.

타입은 `Bolt`, 관계는 `installedIn`과 `manufacturer`를 사용해. 제조사 값은 문서에 없으니 data에는 추가하지 마.

② RDF와 Shape 초안 · 기대 출력 예시

```
# data.ttl
ex:P1 a ex:Bolt ;
  ex:installedIn ex:A1 .

# shapes.ttl
ex:ManufacturerShape a sh:NodeShape ;
  sh:targetNode ex:P1 ;
  sh:property [
    sh:path ex:manufacturer ;
    sh:minCount 1 ;
    sh:nodeKind sh:IRI
  ] .
```

③ 파일 저장 → RDF 파싱 → pySHACL 실행
다음 실습에서 이 데이터와 조건을 직접 검사한다.

발표자가 작성한 대화·출력 예시 · 실제 LLM 호출 아님 · 접두사는 실습 노트북에 포함

직접 스키마 작성과 표 데이터 변환

> 클래스·속성·IRI를 먼저 정하고 원천의 열을 그 정의에 연결합니다. 편집 도구·변환 도구·검증 도구의 역할을 나누면 작업을 재현하기 쉽습니다.

스키마 먼저 · 편집 작업의 개념도

클래스 계층 / 속성 / 개체

```
Component
└ Fastener
   └ Bolt

installedIn: Component → Assembly
P1: Bolt → installedIn → A1
```

Protégé / WebProtégé에서 클래스·속성·공리를 편집하고 저장한다.

행 데이터에서 그래프까지

작업	도구·산출물
CSV 정리·ID 맞추기	OpenRefine
열 → IRI·관계 매핑	RDF Transform 확장
반복 가능한 변환	Python + RDFLib
RDF·Shape 직접 작성	텍스트 편집기 + Turtle
조건 검사	pySHACL + 보고서

UI는 기능 설명용 재구성이다. 설치·실행 캡처가 아니며 RDF 확장 호환성은 확인해야 한다.

출처: Protégé / WebProtégé · OpenRefine 확장 목록 · RDFLib

노트북 실습 — 데이터와 Shape 준비

- P1의 기록과 제조사 필수 조건을 각각 RDF 그래프로 읽는다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [2] data

```
DATA = """ex:P1 a ex:Bolt ;
    ex:installedIn ex:A1 ."""
data = Graph().parse(data=PREFIX + DATA, format="turtle")
print(f"데이터: {len(data)}개 트리플")
```

Out [2]

```
데이터: 2개 트리플
```

In [3] shape

```
SHAPE = """ex:ManufacturerShape a sh:NodeShape ;
    # 이 검사는 P1의 제조사 기록만 확인한다.
    sh:targetNode ex:P1 ;
    sh:property [
        sh:path ex:manufacturer ;
        # 값이 하나 이상이고 IRI인지 검사한다.
        sh:minCount 1 ;
        sh:nodeKind sh:IRI ;
        sh:message "제조사 식별자를 하나 이상 기록하세요."
    ] ."""
shapes = Graph().parse(data=PREFIX + SHAPE, format="turtle")
print(f"Shape: {len(shapes)}개 트리플")
```

Out [3]

```
Shape: 7개 트리플
```

▶ 셀의 실제 저장 출력 표시

노트북 실습 — 검사와 보고서 확인

- `conforms`는 조건 충족 여부, `report`는 위반 근거, `report_text`는 텍스트 보고서다.

manufacturing-shacl.ipynb · Python 3 저장된 실행 결과 재생

In [4] validate

```
conforms, report, report_text = validate(
    data, shacl_graph=shapes, inference="none")
print("conforms =", bool(conforms))
```

Out [4]

```
conforms = False
```

In [5] report

```
for result in report.subjects(RDF.type, SH.ValidationResult):
    print("대상:", report.value(result,
SH.focusNode).n3(data.namespace_manager))
    print("속성:", report.value(result,
SH.resultPath).n3(data.namespace_manager))
    print("조건:", str(report.value(result,
SH.sourceConstraintComponent)).split("#")[-1])
    print("메시지:", report.value(result, SH.resultMessage))
```

Out [5]

```
대상: ex:P1
속성: ex:manufacturer
조건: MinCountConstraintComponent
메시지: 제조사 식별자를 하나 이상 기록하세요.
```

▶ 셀의 실제 저장 출력 표시